**Technik der MMI**, UXD-B
Praktikum, WiSe 2026/27

**Praktikumsblatt 2**
Prof. Dr. Andreas Riener, Prof. Dr. Simon Nestler

# Visuelle Programmierung mit vvvv

### Worum geht es in diesem Blatt?

Sie vertiefen die Arbeit mit der visuellen Programmierumgebung vvvv gamma. Sie bauen Patches von Grund auf, nutzen die Maus als eindimensionalen Positionsgeber, ersetzen sie durch Joystick, Leap Motion oder Kinect und entwickeln daraus eine Anwendung, mit der Sie Punkte von Interesse in einem Hintergrundbild ansteuern. Sie programmieren, indem Sie Knoten verbinden, nicht indem Sie Code schreiben.

### Was erwarten wir von der Dokumentation (Protokoll)?

- Beantworten Sie die gestellten Fragen in einem Protokoll (Moodle-Fragebogen/Quiz)
- Machen Sie zusätzlich Screenshots Ihrer Patches/Programme und Fotos, die Ihre Lösungen in „Aktion“ zeigen (vvvv-Screenshots legen Sie direkt in vvvv mit der Tastenkombination „**STRG + 2**“ in der Zwischenablage ab); vergessen Sie nicht, die Patches zu speichern!
- Halten Sie Ihre **eigene Interpretation** fest: Was haben Sie im Experiment erkannt, was ist Ihnen aufgefallen?
- Welche **Probleme** sind Ihnen begegnet, und wie haben Sie sie **gelöst**?

### Wie?

- **Bearbeiten Sie zur Abgabe den Test „Abgabe Praktikumsblatt 2“** in Moodle (Kursraum Technik der Mensch-Maschine Interaktion).
- **Kopieren Sie dazu Ihre Antworten, Screenshots, Fotos usw. in die jeweiligen Antwortfelder.** Das Protokoll können Sie in der Praktikumsstunde direkt im Angabe-Worddokument oder in einem separaten Textdokument schreiben.
- **Speichern Sie die vvvv-Patches und geben Sie sie als Anhänge mit ab!**
- In Zweiergruppen lädt nur die protokollführende Person hoch; im Protokoll stehen beide mit vollständigem Namen.
- **ACHTUNG: Chromium-basierte Browser können offenbar keine Bilder kopieren. Nutzen Sie in dem Fall einen anderen Browser, z. B. Firefox.**

### Abgabe?

- **Abgabe innerhalb von 48 Stunden** nach Ihrem Praktikumstermin, in Moodle

(Ab) heute arbeiten Sie mit der visuellen Programmiersprache bzw. -umgebung „*vvvv gamma*“ (auch „v4“ genannt). „*vvvv*“ (`http://www.vvvv.org/download/`) entstand als Werkzeug, mit dem auch „Nicht-Programmierer“ ansprechende Programme und Installationen entwickeln können: Interaktionsdesigner „verstehen” abstrakten Programmcode oft nicht. Optimal also auch für UXD'ler ;-)

Mit „*vvvv*“ entwickeln Sie Prototypen auf bildlicher Ebene: Sie programmieren mit Komponenten und verbinden sie mit „*Links*“. „*vvvv*“ ist frei verfügbar, und die Community rund um „*vvvv*“ ist sehr groß. Deshalb gibt es Hunderte von Interaktionskomponenten (sogenannte „*Nodes*“), die Sie in eigenen Programmen wiederverwenden können, z. B. für die Interaktion mit Kinect, Leap Motion usw.

**Nutzen Sie zu diesem Aufgabenblatt auch die Powerpoint-Folien „Einführung in vvvv“ (und die zugehörigen vvvv-Patches). Sie erklären Schritt für Schritt (Tutorial-Stil) die Basisinformationen zu vvvv, Grundbegriffe, erste Programme usw. Den Umgang mit „vvvv“ erleichtern außerdem diese Ressourcen:**

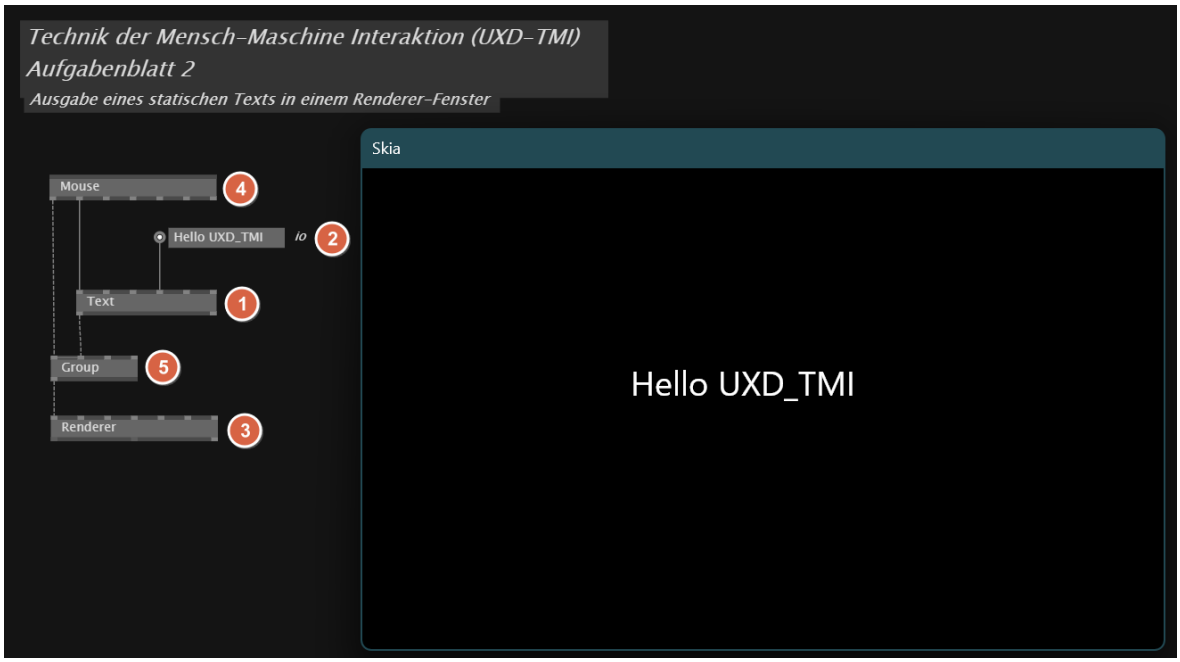
- Getting Started (Grundlagen zu vvvv gamma mit Links zu Video-Tutorials): `https://thegraybook.vvvv.org/reference/getting-started/overview.html`
- Übersicht über die Tastaturbelegung: `https://thegraybook.vvvv.org/reference/hde/keyboard-shortcuts.html`

## 1) Erste Schritte mit „vvvv“

Sie beginnen mit zwei „*vvvv*“-Programmen (=„*Patches*“) aus dem „*vvvv*“-Tutorial der Vorlesung. Damit verschaffen Sie sich (wieder) einen ersten Überblick, wie „vvvv“ funktioniert und sich verhält.

### 1.a) „Hello World“

Dieser „*vvvv*“-Patch gibt zunächst nur statischen Text aus. Dann fügen Sie dynamisches Verhalten hinzu, bis sich der Text mit der Maus bewegt. Eine Beschreibung finden Sie auch im „*vvvv*“-Tutorial.




**Vorgangsweise**

- Starten Sie „*vvvv gamma*“.
- Fügen Sie nach und nach die einzelnen Komponenten („*Nodes*“) wie folgt ein:

**Hinweis: Die Nummern in Klammern entsprechen den Zahlen in den Kreisen der Patch-Screenshots**

(1) Ein Doppelklick (links) in das Fenster öffnet den „*NodeBrowser*“. Er listet alle verfügbaren „*Nodes*“ (Komponenten) auf; über das Eingabefeld können Sie filtern. Suchen Sie hier zuerst nach „Skia“. Skia ist die 2D-Grafik-Library (Render-Engine). Wenn Sie vvvv gamma zum ersten Mal starten, ist diese Bibliothek möglicherweise noch nicht eingebunden; der NodeBrowser zeigt sie dann mit einem Paketsymbol an. Weil Skia eine Standard-Library ist, binden Sie sie mit einem einfachen Mausklick ein. Danach zeigt der NodeBrowser Skia mit einem Ordnersymbol an. Klicken Sie unter Skia auf den Ordner „Text“ und dann auf den Node „`Text [Graphics.Skia.Layers.Text]`“; damit fügen Sie ihn in Ihren Patch ein.

(2) Fügen Sie einen Node vom Typ „`IOBox (String)`“ ein, doppelklicken Sie mit der linken Maustaste auf den Node und geben Sie den Text „Hello World“ ein. Vergrößern Sie den Node, wenn Sie den eingegebenen Text im Patch nicht vollständig sehen. Verbinden Sie den Output-Pin dieses Nodes (2) mit dem 4. Eingabe-Pin des „*Text*“-Nodes (1).

(3) Fügen Sie einen „`Renderer (Graphic.Skia)`“-Node ein (es öffnet sich ein neues Skia Renderer-Fenster) und verbinden Sie den Output-Pin des „*Text*“-Nodes (1) mit dem 1. Input-Pin des „*Renderer*“-Nodes (3). Im Fenster erscheint bereits der eingegebene Text. Speichern Sie den Patch mit „STRG + SHIFT + S“ als „`HelloWorld.vl`“ ab.

(4) Damit sich der Text mit der Maus bewegt, fügen Sie zuerst einen Node vom Typ „`Mouse [Graphics.Skia.IO]`“ ein und verbinden seinen Output-Pin „Position in World“ mit dem Eingabe-Pin „Position“ des „*Text*“-Nodes (1). Sie werden feststellen: Der Text bewegt sich im Renderer-Fenster noch nicht, wenn Sie die Maus innerhalb des Renderer-Fensters bewegen.

(5) Dafür fügen Sie noch einen „`Group [Graphics.Skia]`“-Node ein. Verbinden Sie den Ausgang dieses Nodes mit dem Input-Pin des „*Renderer*“-Nodes (3). Den Ausgang des „*Text*“-Nodes (1) verbinden Sie mit einem der Input-Pins des „*Group*“-Nodes, den Context-Ausgang des „*Mouse*“-Nodes (ganz links) mit einem weiteren Input-Pin des „*Group*“-Nodes.

– Speichern Sie den Patch mit Dateinamen „`HelloUXD_TMI.vl`“ ab (STRG + SHIFT + S).

**Frage 1:** a) Welche Funktion könnte der Ausgang „Context“ des „*Mouse*“-Nodes haben? b) Wofür braucht der Patch den „*Group*“-Node?

*Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).*

– Ergänzen Sie weiteres dynamisches Verhalten: Der Text soll **zusätzlich** (zur Bewegung mit der Maus) automatisch rotieren. Hinweis: Sie brauchen dazu (mindestens) Nodes vom Typ „`TransformSRT [Graphics.Skia.Transform]`“ und „`LFO [Animation]`“. Speichern Sie den Patch als „`HelloUXD_TMI_Rotate.vl`“ ab.

**Frage 2:** a) Wie bzw. wo stellen Sie die Periodendauer der Rotation (d. h. die Rotationsgeschwindigkeit) ein? b) Wofür lassen sich die beiden Input-Pins „Scaling“ und „Translation“ des „*TransformSRT*“-Nodes verwenden?

*Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).*

### 1.b) Maussteuerung mit „vvvv“

Das 2. Beispiel finden Sie (in ähnlicher Form) ebenfalls im „*vvvv*“-Tutorial. Es zeigt, wie einfach Sie in „*vvvv*“ aus beliebigen Patches auf externe Eingabegeräte zugreifen und damit interaktive Programme erstellen.

Was die einzelnen Komponenten genau machen, müssen Sie hier noch **nicht vollständig verstehen**. Arbeiten Sie dieses „Mini-Tutorial“ einfach Schritt für Schritt wie unten beschrieben durch. Ob die Verbindungslinien („*Links*“ zwischen Nodes) gerade bzw. rechtwinklig sind, ist ebenfalls egal; für visuelle Verbesserungen ist später noch Zeit…

Der Patch in der Abbildung und in der Beschreibung unten erfüllt dieselbe Funktion wie der im Tutorial, verarbeitet aber leicht anders. So sehen Sie, dass es in „vvvv“ meist mehrere Wege gibt, ein Problem zu lösen.

**Vorgangsweise**

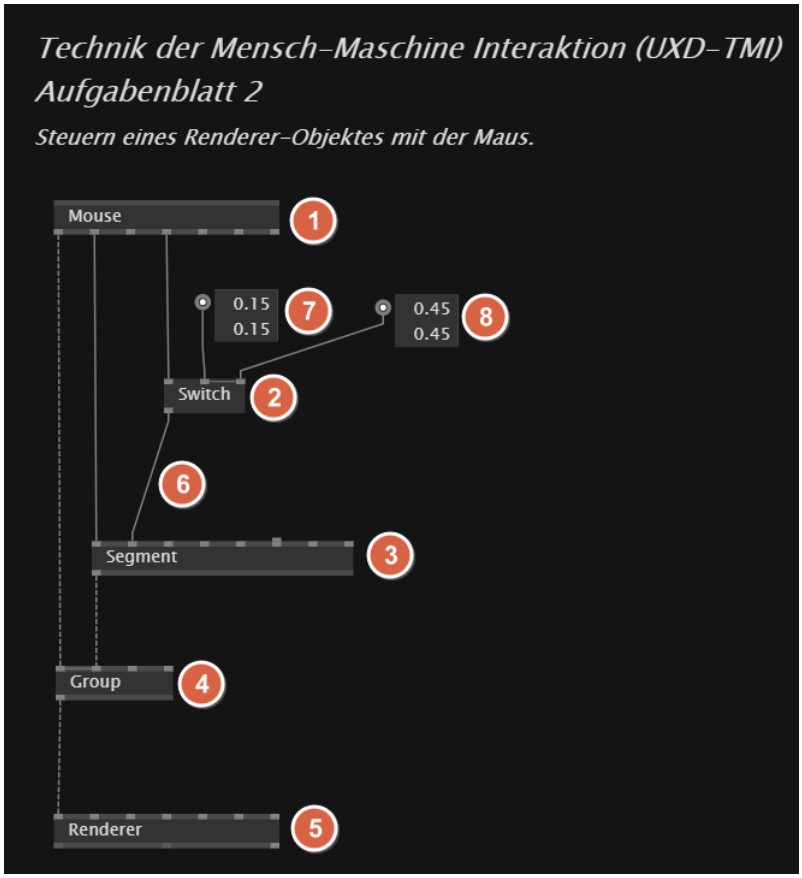
– Starten Sie „*vvvv*“ und legen Sie einen neuen, leeren Patch an (New → VL Document).

(1) Ein Doppelklick (links) in das Fenster öffnet den „*NodeBrowser*“. Er listet alle verfügbaren „*Nodes*“ (Komponenten) auf; über das Eingabefeld können Sie filtern. Suchen Sie hier den „`Mouse [Graphics.Skia.IO]`“-Node und fügen Sie ihn in Ihren Patch ein. (Denken Sie daran: Eventuell müssen Sie zuerst das Paket (Nuget) VL.Skia mit einem Klick referenzieren, damit der Node verfügbar ist.)

– Das allgemeine Hilfesystem von v4 rufen Sie mit der Taste „**F1**“ auf (es darf kein Node ausgewählt sein) bzw. über das „Quad menu“ → „Help Browser F1“. Darin fragen Sie auch Informationen zu einzelnen Nodes ab: „Help Browser“ → „Learn“ → „Node Info“.

– Für viele Nodes gibt es eine eigene **Hilfeseite mit Beispielpatches**. **Wählen Sie mit einem Linksklick einen Node in einem Patch aus; „F1“ öffnet die passende Hilfeseite.** Probieren Sie das aus: Markieren Sie z. B. den „*Mouse*“-Node und drücken Sie „F1“. Die Hilfeseite sollte sich öffnen. Schließen Sie das Fenster wieder über das „Active Document Menu“ (→ linke Maustaste auf den Patchnamen im Fenstertitelbereich → „Close“ bzw. X) oder mit der Tastenkombination „STRG + W“.

– **Hinweis**: Die einzelnen Nodes konfigurieren und überwachen Sie am leichtesten im „**Inspector**“ („Quad menu“ → Windows → Inspector bzw. Tastenkombination „STRG + I“). Halten Sie dieses Fenster während der Arbeit an Patches immer geöffnet.



– Weitere Elemente fügen Sie mit einem Doppelklick links in das „*vvvv*“-Programmfenster ein. Sehen Sie sich für jedes eingefügte Element a) die Hilfeseite an, b) die Ein-/Ausgänge, die der Node anbietet, und c) die erweiterten Konfigurationsmöglichkeiten im „Inspector“. So verstehen Sie die einzelnen Komponenten besser (Informationen zu den einzelnen Komponenten finden Sie auch im Foliensatz zum vvvv-Tutorial).

(2) Node-Typ „`Switch [Control]`“, mit dem „*Mouse*“-Node verbinden (wie oben dargestellt).

(3) Node-Typ „`Segment [Graphics.Skia.Layer]`“. Verbinden Sie den Output-Pin „Position in World“ des „*Mouse*“-Nodes mit dem Input-Pin „Position“ des „*Segment*“-Nodes. Den Output-Pin des „*Switch*“-Nodes verbinden Sie zunächst noch **nicht** mit dem „*Segment*“-Node.

(4) Node-Typ „`Group [Graphics.Skia]`“; verbinden Sie den Context-Pin des „*Mouse*“-Nodes und den Output-Pin des „*Segment*“-Nodes mit je einem Eingang des „*Group*“-Nodes.

(5) Node-Typ „`Renderer [Graphics.Skia]`“; es öffnet sich ein neues Renderer-Fenster. Verbinden Sie den Output-Pin des „*Group*“-Nodes mit dem Input-Pin des Renderers.

– Speichern Sie den Patch als „`Maussteuerung.vl`“ ab (über das Document-Menü oder mit STRG + SHIFT + S).
– Im Renderer-Fenster sollte jetzt ein weißes Objekt erscheinen, das Sie mit der Maus steuern können…

**Frage 3:** Was fällt Ihnen auf?

*Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).*

– Öffnen Sie den „Inspector“ (falls er nicht schon offen ist), wählen Sie den „*Renderer*“-Node (5) aus und ändern Sie den Wert des Attributes „Clear“ (linke Maustaste). Alternativ klicken Sie mit der rechten Maustaste auf den Pin „Clear“ des „*Renderer*“-Nodes. Jeder Klick ändert den internen Zustand des Eingangs.

**Frage 4:** Was stellen Sie bei „Clear“ im Renderer-Fenster fest?

*Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).*

– Ändern Sie im „Inspector“ den Wert „Count“ des „*Segment*“-Nodes (3) auf 1, 2, 3, 4, 5, 7, 10, 20, 75.

**Frage 5:** Was stellen Sie fest, wenn Sie „Count“ ändern?

*Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).*

– Vermutlich haben Sie schon bemerkt: Mausklicks ändern die Größe des weißen Kreises (noch) nicht. Fügen Sie dafür die Komponenten (6), (7) und (8) ein:

(6) Verbinden Sie den Output-Pin des Nodes „`Switch [Control]`“ mit dem Eingang „Size“ des „*Segment*“-Nodes.

(7) Doppelklicken Sie auf den ersten Input-Pin des „*Switch*“-Nodes und wählen Sie im NodeBrowser, der sich öffnet, eine „IOBox“ aus. An diesem Pin sollte dann eine passende IOBox (Datentyp Vector2) hängen. Doppelklicken Sie links auf die „*IOBox*“ und geben Sie in beide Zeilen den Wert „0.15“ ein (oder nutzen Sie das Feld, das über der IOBox erscheint; darüber geben Sie denselben Wert für beide Elemente ein).

(8) Wiederholen Sie dieselben Schritte wie in (7) mit dem zweiten Input-Pin des „*Switch*“-Nodes. Tragen Sie zweimal den Wert „0.45“ in die IOBox ein.

**Frage 6:** Was bemerken Sie (im Renderer-Fenster), wenn Sie die Werte der beiden „*IOBox*“-Nodes (7) und (8) ändern? Welche Werte sind als Eingabe erlaubt bzw. sinnvoll? Was geschieht, wenn die beiden Werte in einer der IOBoxen nicht identisch sind?

*Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).*

**Frage 7:** Was müssen Sie am Patch ändern, damit die mittlere statt der linken Maustaste die Größe des Kreises verändert? Wie müssen Sie den Patch verändern, damit das Objekt bei einem Mausklick „verschwindet“ (und sonst wie zuvor der Mausbewegung folgt)?

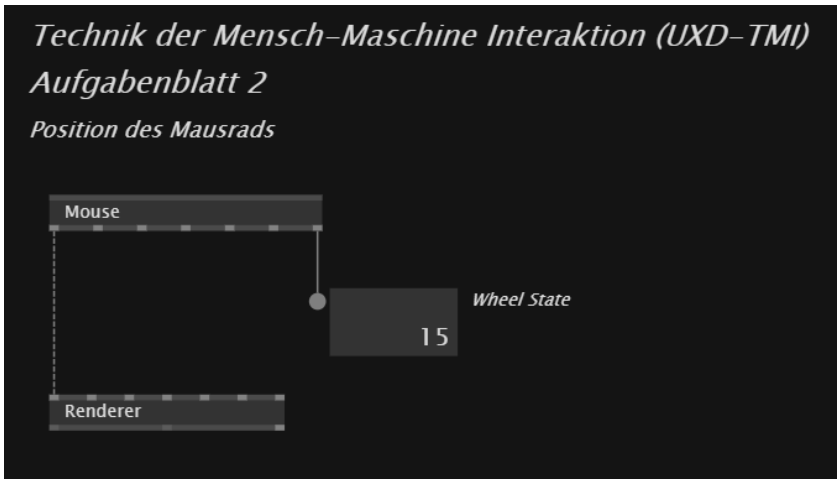
*Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).*

## 2) Verwendung der Maus als eindimensionaler Positionsgeber

Über den „`Mouse [Graphics.Skia.IO]`“-Node greifen Sie in „*vvvv*“ ganz einfach auf die Maus zu, das haben Sie gerade gesehen. Beachten Sie: Der Node erfasst (und verarbeitet) die Mausposition nur, wenn die Maus **innerhalb eines Renderer-Fensters** ist. Wie oben gesehen, hat der „`Mouse [Graphics.Skia.IO]`“-Node mehrere Ausgabe-Pins: Neben der X- und Y-Position der Maus gibt er auch alle drei Tasten und die Position des Mausrades aus. Als Nächstes sehen Sie, wie (bzw. wozu) sich die **Position des Mausrades** in einer eigenen Applikation nutzen lässt…

### 2.a) Darstellung der Position des Mausrads

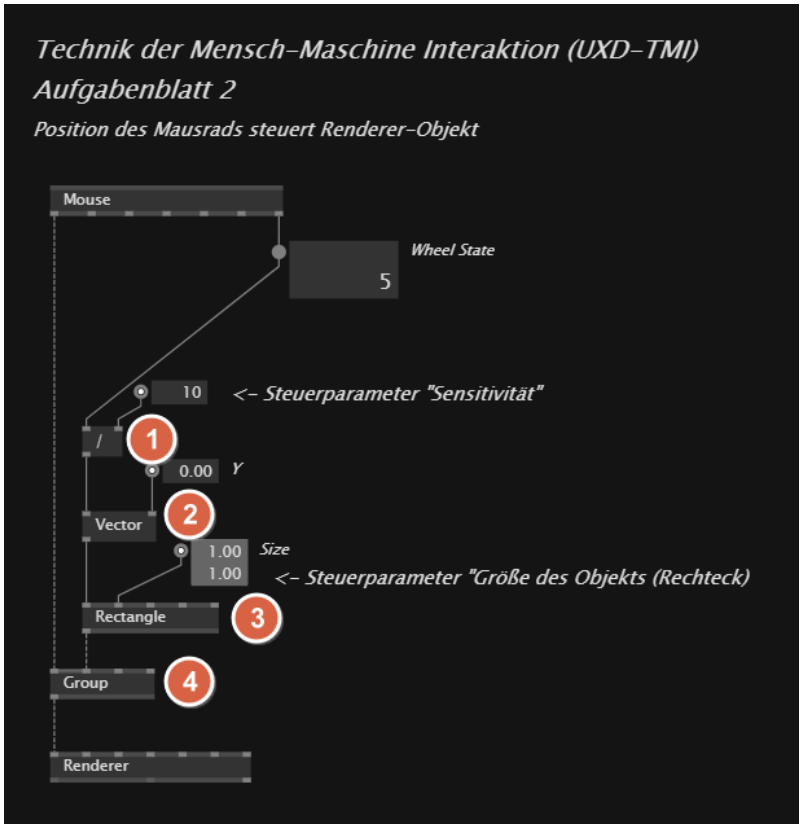
Das Mausrad lässt sich unendlich lange in eine Richtung drehen. Sein Wert wird damit beliebig groß oder klein: Je länger Sie in eine Richtung drehen, desto weiter steigt oder fällt er, ohne Ende…




**Vorgangsweise**

- Starten Sie „*vvvv gamma*“ und legen Sie einen neuen, leeren Patch an
- Fügen Sie über den „*NodeBrowser*“ (Doppelklick links) einen „`Mouse [Graphics.Skia.IO]`“-Node in Ihren Patch ein
- Starten Sie einen Link am Ausgang „Wheel State“ und schließen Sie ihn mit einem Mittelklick ab. Dort sollte eine „`IOBox (Integer 32)`“ entstehen. **Hinweis: Achten Sie immer auf die Bezeichnung der Pins eines Nodes. Ihre Position (hier z. B. ganz rechts) kann je nach Konfiguration von Rechner zu Rechner verschieden sein!**
- Fügen Sie einen „`Renderer [Graphics.Skia]`“ ein und verbinden Sie den Context-Ausgang des „*Mouse*“-Nodes mit dem Input-Pin des Renderers.
- Speichern Sie den Patch als „`Mausrad1.vl`“ ab und machen Sie (bei ausgewähltem Patch-Fenster) mit „STRG + 2“ oder mit der Druck-Taste einen Screenshot Ihres Patches. *Kopieren Sie den Screenshot in Ihr Protokoll.*
- Wenn Sie jetzt im Renderer-Fenster das Mausrad drehen, zeigt die „*IOBox*“ seinen aktuellen Wert an. Sie werden sehen: Der Wert kann beliebig groß oder klein werden… (Sie können den Wert auch im „Inspector“ mitverfolgen; vergessen Sie nicht, die „*IOBox*“ auszuwählen!)

Im nächsten Schritt erweitern Sie den Patch, damit der Mausrad-Wert ein Objekt im Renderer steuert.




**Vorgangsweise**

- Basis ist der Patch „`Mausrad1.vl`“ (speichern Sie ihn als „`Mausrad2.vl`“ ab).
- Danach bauen Sie das vvvv-Programm (die Blockschaltung) Schritt für Schritt auf:

(1) Damit Sie die **Sensitivität des Mausrads** verändern können, fügen Sie einen Divisions-Node („`/ [Primitive.Float32]`“) ein und verbinden ihn mit dem Ausgang „Wheel State“ des „*Mouse*“-Nodes. Am zweiten Eingang des Divisions-Nodes erstellen Sie einen Link und eine IOBox. Tragen Sie in die IOBox zunächst den Wert „10.0“ ein.

(2) Node-Typ „`Vector (Join) [2D.Vector2]`“; verbinden Sie den ersten Eingang mit dem Ergebnis-Pin des Divisionsoperators (1). Erzeugen Sie am zweiten Eingang einen Link und eine IOBox. Belassen Sie den Wert in der IOBox bei „0.0“ (alternativ setzen Sie den Wert direkt im „Inspector“).

(3) Node-Typ „`Rectangle [Graphics.Skia.Layers]`“ zum Zeichnen eines Rechtecks im Renderer-Fenster. Erstellen Sie am Eingang „Size“ einen Link mit einer IOBox.

(4) Fügen Sie einen Node vom Typ „`Group [Graphics.Skia]`“ zwischen den Nodes „*Mouse*“ und „*Renderer*“ ein. Verbinden Sie einen weiteren Eingang des „*Group*“-Nodes mit dem Output des „*Rectangle*“-Nodes.

- Speichern Sie den Patch ab.
- Wenn Sie jetzt in das Renderer-Fenster klicken, verschieben Sie das Quadrat bzw. Rechteck mit dem Mausrad nach links und rechts…



**Frage 8:** a) Was macht der Divisionsfaktor (Steuerparameter „Sensitivität“)? b) Was beeinflusst der Wert „Y“ des „*Vector*“-Nodes, siehe Nummer (2) oben?

*Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).*

**Frage 9:** a) Was stellen Sie (im Renderer) fest, wenn Sie die Werte der IOBox „Size“, siehe Nummer (3) oben, ändern (probieren Sie auch unterschiedliche Werte für die beiden Zahlen aus)? b) Welches „Problem“ stellen Sie bei der „Navigation mit dem Mausrad“ fest? Ist das so gewünscht, und welches Verhalten würden Sie sich im Normalfall für eine Anwendung wünschen?

*Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).*

Das Mausrad verhält sich hier problematisch: Für eine Positionsangabe müssen Sie nicht nur den aktuellen Wert der Maus kennen, sondern auch die möglichen Extremwerte. Damit Sie die Position der Maus im aktuellen Fenster besser kontrollieren können, bauen Sie in der nächsten Aufgabe eine kleine Hilfe in den Patch ein.

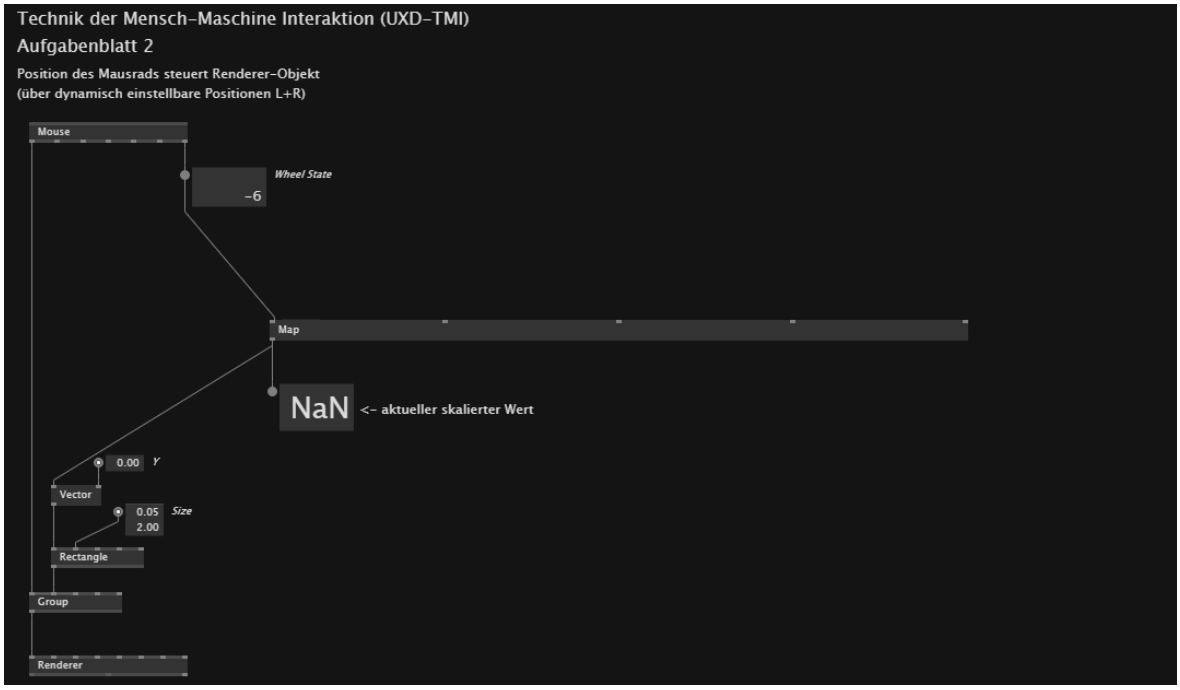
### 2.b) Dynamisch einstellbare Positionen für die Mausradsteuerung

Das Mausrad lässt sich unendlich lange in eine Richtung drehen. Sein Wert wird damit beliebig groß oder klein: Je länger Sie in eine Richtung drehen, desto weiter steigt oder fällt er, ohne Ende…

Die Idee (siehe Skizze unten): Das Mausrad der „umgedrehten“ Maus dient als eindimensionaler Positionsgeber. Die Markierungen „linker Rand“ und „rechter Rand“ (gedacht als Barriere) am Schreibtisch sollen sich dynamisch verändern lassen. Der Bereich dazwischen wird so auf das Renderer-Fenster „gemappt“, dass der „linke Rand“ am Schreibtisch mit dem linken Rand des Renderer-Fensters übereinstimmt (analog für den rechten Rand).

Der nächste Patch soll sich so verhalten (d. h., das implementieren Sie in „vvvv“): Sie drehen die Maus an einer beliebigen Stelle am Schreibtisch um (siehe Abbildung auf der nächsten Seite) und bewegen die umgedrehte Maus an den „linken Rand“; das Mausrad liegt dabei ständig auf der Schreibtischplatte auf. Dann drücken Sie eine Taste auf der Tastatur (hier: „Pfeil links“) und markieren diesen Punkt so als linken Rand. Danach „fahren“ Sie mit der Maus an den „rechten Rand“ (das Mausrad liegt wieder ständig auf und dreht sich mit) und drücken eine weitere Taste („Pfeil rechts“). Damit markieren Sie diesen Punkt als größten Wert und legen zugleich den rechten Rand im Renderer-Fenster fest.




Technik der Mensch-Maschine Interaktion (UXD-TMI)
Aufgabenblatt 2
Position des Mausrads steuert Renderer-Objekt
(über dynamisch einstellbare Positionen L+R)
Mouse
Wheel State
-6
Map
NaN
<- aktueller skalierter Wert
0.00
Y
Vector
0.05
2.00
Size
Rectangle
Group
Renderer

**Vorgangsweise**

- Starten Sie mit dem Patch „`Mausrad2.vl`“ und speichern Sie ihn als „`Mausrad3.vl`“ ab.
- Die Grundidee: Sie mappen den Bereich zwischen linkem und rechtem Rand (am Schreibtisch und im Renderer-Fenster) jeweils auf den Wertebereich des Koordinatensystems des Renderer-Fensters. Dazu ändern und ergänzen Sie den Patch so:

(1) Schieben Sie den oben **gelb gestrichelt eingezeichneten Bereich** nach unten und schaffen Sie Platz im Patch. Fügen Sie einen neuen „`Map [Math.Ranges]`“-Node ein (am besten wie oben im rechten Fensterbereich). Vergrößern bzw. verbreitern Sie den „`Map [Math.Ranges]`“-Node; so haben Sie später mehr Platz, um weitere Nodes einzufügen und an die einzelnen Eingabe-Pins anzuschließen. Die Abbildung oben zeigt den Zwischenstand. Den Divisions-Node samt der IOBox für die Eingabe können Sie entfernen.

(2) Verbinden Sie den Context-Ausgang des „*Mouse*“-Nodes wieder mit dem „Input“ des „*Group*“-Nodes und erweitern Sie den „*Group*“-Node um den „Input 3“: Node markieren und STRG + PLUS drücken.

Ergänzen Sie den Patch dann wie in der Abbildung unten:

(3) Erstellen Sie eine „`IOBox [Float32]`“ am Eingang „Output Minimum“ des „*Map*“-Nodes. Verbinden Sie die IOBox mit diesem Eingang und geben Sie zunächst den Wert „-1.0“ ein (**um einen korrekten Wert kümmern Sie sich später!**).

(4) Erstellen Sie auf dieselbe Weise einen Link mit IOBox und Wert „1.0“ am Eingang „Output Maximum“.

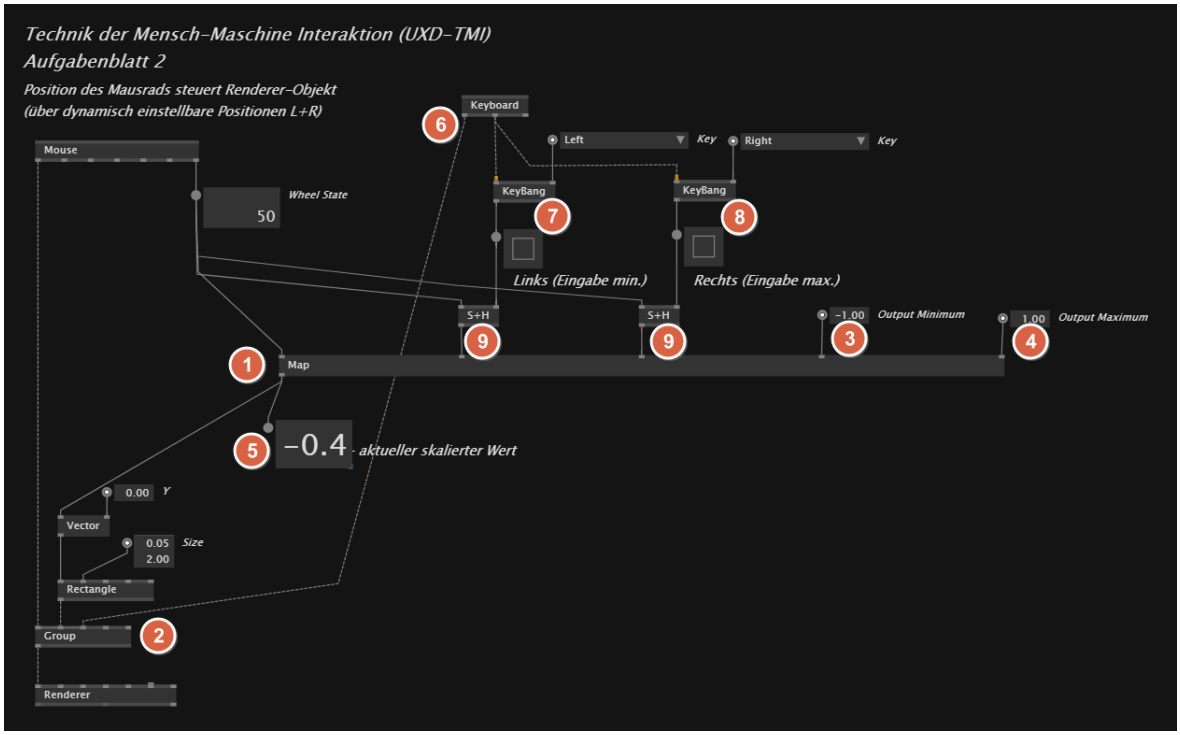
(5) Erstellen Sie am Ausgang des „*Map*“-Nodes einen Link mit einer IOBox. Wenn Sie mit der mittleren Maustaste auf den Wert in der IOBox klicken, können Sie das Attribut „Font Size“ auf „20“ setzen. An diesem Node kontrollieren Sie, ob die Skalierung korrekt funktioniert.

(6) Fügen Sie einen Node vom Typ „`Keyboard [Graphics.Skia.IO]`“ ein, verbinden Sie seinen Context-Ausgang mit „Input 3“ des „*Group*“-Nodes, und fügen Sie die folgenden Nodes ein:

(7) Node-Typ „`KeyBang [Graphics.Skia.IO.Keyboard]`“ (KeyBang: Tasten werden nur beim Drücken/bei absteigender Flanke ausgewertet). Verbinden Sie den Output-Pin „Keyboard“ von (6) mit dem Eingabe-Pin „Keyboard Device“ dieses Nodes (er prüft, ob eine der festgelegten Tasten gedrückt wurde). Die zu überwachende Taste legen Sie entweder im „Inspector“ unter „Key“ fest oder wählen sie über einen Link mit einer IOBox aus. Hier ist es die linke Pfeiltaste; wählen Sie also unter „Key“ „Left“ aus.

(8) Fügen Sie einen weiteren Node vom Typ „`KeyBang [Graphics.Skia.IO.Keyboard]`“ ein, verbinden Sie die Eingangs-Pins wie unter (7) und setzen Sie den Wert (Drop-down-Menü) auf die rechte Pfeiltaste „Right“.

(9) Fügen Sie zwei „`S+H [Control]`“-Nodes ein und stellen Sie die Verbindungen her. Ergebnis:

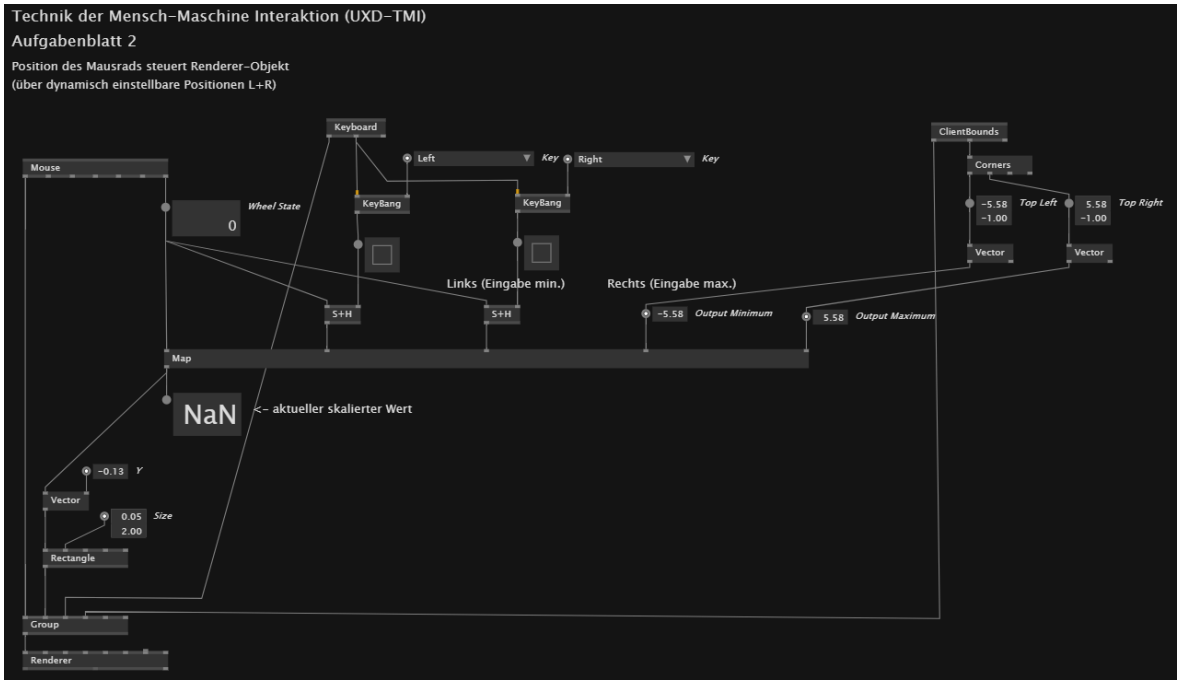



- Im Ausgabefenster soll statt des Quadrats künftig eine Art „Fadenkreuz“ die Position des Mausrads zeigen (vereinfacht: ein vertikaler Strich). Dazu setzen Sie in der IOBox „Size“ am Node „`Rectangle [Graphics.Skia.Layers]`“ die Eingangsparameter z. B. auf „0.05“ und „2.0“ (den Wert Y legen Sie auf „0.0“ fest).
- Speichern Sie den Patch und testen Sie ihn:
  - Machen Sie 2 Markierungen auf dem Schreibtisch, z. B. mit Post-its. Bitte nicht mit Bleistift, schon gar nicht mit Faserschreiber ;-)
  - Klicken Sie am Bildschirm in das Renderer-Fenster
  - Drehen Sie die Maus um, fahren Sie mit dem Mausrad an Ihre linke Markierung, drücken Sie die „Pfeiltaste links“, fahren Sie mit der Maus/dem Mausrad bis zur rechten Markierung (Post-it) und drücken Sie die „Pfeiltaste rechts“.
- Jetzt müsste der Bereich zwischen linker und rechter Markierung am Tisch genau auf den linken bzw. rechten Rand des Renderer-Fensters gemappt sein… Sie werden feststellen, dass das nicht der Fall ist.

**Frage 10:** Was könnte Ihrer Meinung nach das Problem sein? (Hinweis: Es hat einerseits mit dem Koordinatensystem des Renderer-Fensters zu tun und andererseits mit den beiden Werten für Output Minimum „-1.0“ und Output Maximum „1.0“.) Wie könnte bzw. müsste man das Problem beheben?

*Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).*

Die Lösung zu Frage 10 finden Sie als Erweiterung zum Patch im Datentausch-Laufwerk „P:“ als „`Mausrad3 ClientBounds.vl`“. Kopieren Sie den gelb umrandeten Bereich in Ihren Patch und vervollständigen Sie die Links wie unten dargestellt:

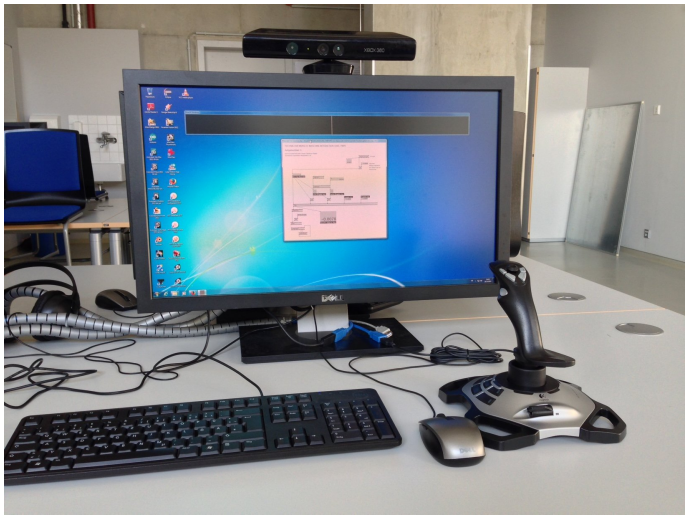



Speichern Sie Ihren Patch als „`Mausrad4.vl`“ ab und testen Sie ihn wieder wie oben beschrieben (linken/rechten Rand bei umgedrehter Maus festlegen…).

**Frage 11:** a) Welches (unpraktische) Problem stellen Sie weiterhin fest? Was ließe sich dagegen tun? b) Was bedeutet es, wenn der Wert in der „`IOBox (Float32)`“ mit der Bezeichnung „aktuell skalierter Wert“ kleiner als Output Minimum oder größer als Output Maximum wird? c) Wie stellen Sie in der aktuellen Schaltung die „Sensitivität“ ein (d. h. die Geschwindigkeit, mit der sich der „Cursor“ am Bildschirm bewegt, bzw. die Feinheit der Steuerung)?

*Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).*

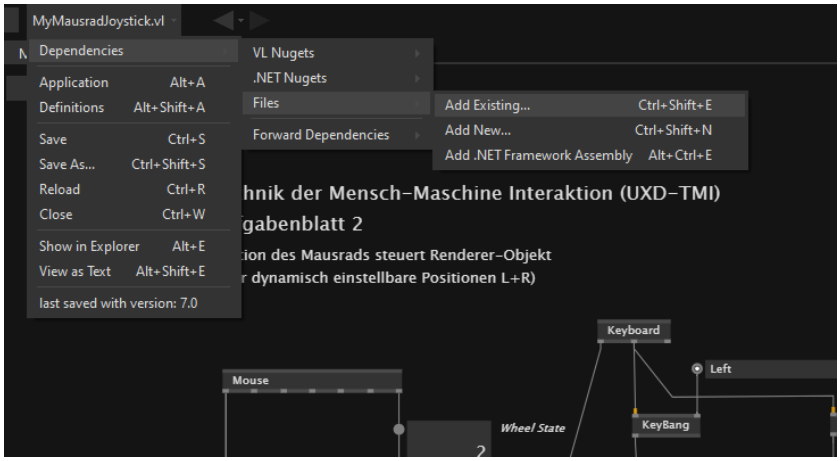
## 2.c) Alternative Eingabegeräte: Joystick, „Leap Motion“, „Kinect“



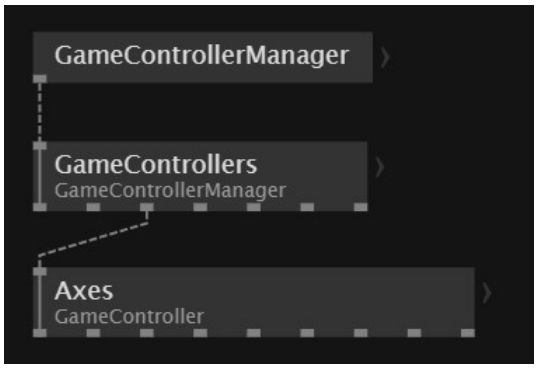
Mit dem Patch „`Mausrad4.vl`“ nutzen Sie jetzt die Maus (bzw. das Mausrad) als **linearen Positionsgeber**. Das funktioniert grundsätzlich sehr gut, ist aber wohl etwas „uncool“. Kein Problem: In „vvvv“ ersetzen Sie die Maus sehr einfach durch andere Eingabesensorik, etwa Joysticks oder Gestenhardware (Leap Motion, Kinect). Wie das geht, sehen Sie hier (kurz).

In vvvv gamma ist die Unterstützung externer Hardware leider noch nicht ganz so weit wie in der Vorgängerversion vvvv beta. Mit etwas Hilfestellung sollte es aber gelingen, die Mausradsteuerung durch Gesten oder Joystickbewegungen zu ersetzen (für „visuelle Programmierer“ mit Grunderfahrung in „vvvv“…).

- Ausgangspunkt ist der Patch „`Mausrad4.vl`“. Erstellen Sie davon (im Windows Explorer) drei Kopien: „`MausradJoystick.vl`“, „`MausradLeap.vl`“ und „`MausradKinect.vl`“.
  - **Joystick:** Die Library „GameController“ ist noch „work in progress“ (WIP) und lässt sich nicht über Nuget installieren. Stattdessen laden Sie eine ZIP-Datei mit einer „`GameController.vl`“ und einer Hilfe-VL von `https://forum.vvvv.org/uploads/short-url/8XfEbHEdsyEqspM8oPVKpgVbM0E.zip` herunter. Die „`GameController.vl`“ binden Sie (nach dem Entpacken) ein: Klick auf den Patch-Namen (des außenliegenden Fensters) → „Dependencies“ → „File“ → „Add Existing...“. Danach stehen die passenden Nodes im NodeBrowser zur Verfügung und lassen sich im Patch verwenden. Der Thread im vvvv-Forum dazu: `https://discourse.vvvv.org/t/vl-gamecontroller/20570`




Der „*GameController*“-Node braucht einen Renderer vom Typ „Stride“ (nicht „Skia“). Ersetzen Sie deshalb im Patch den „`Renderer [Graphics.Skia]`“ durch einen „`SkiaRender [Stride.Renderering]`“ und ein „`RenderWindow [Stride]`“ (gilt nur für den Joystick!). Achtung: Der Joystick liefert nur Werte bei aktiviertem Renderer-Fenster; vergessen Sie also den Mausklick in das Renderer-Fenster nicht ;-)



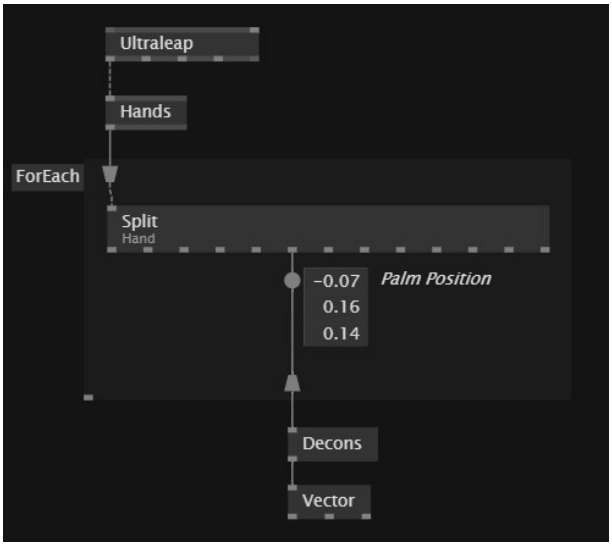




– **Leap Motion:** Damit vvvv den Leap Motion Controller erkennt, muss neben der passenden Ultraleap-Library die Software „Gemini“ von UltraLeap installiert sein (auf den Laborrechnern bereits erledigt). Die vvvv-Library „Ultraleap“ installieren Sie über die Konsole („Quad menu“ → Manage Nugets → Commandline) mit `nuget install VL.Devices.Ultraleap`. Danach binden Sie sie über den NodeBrowser in den Patch ein.

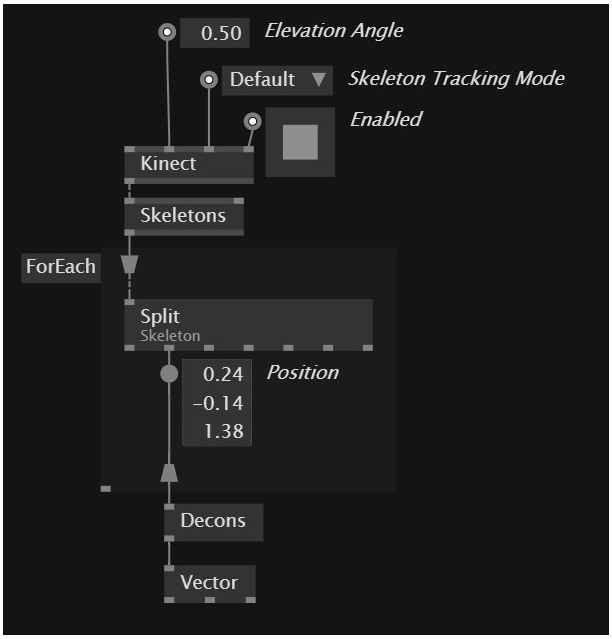
Der „*Hands*“-Node der Ultraleap-Bibliothek liefert einen Spread von Händen (1 oder 2; also getrennte Daten für jede erkannte Hand). Die Skelettdaten extrahieren Sie deshalb in einer Schleife „über alle Hände“ (Kommando ForEach; z. B. rechte Maustaste auf den Node „`Split.Devices.Ultraleap.Hand`“ und „Surround“). Die Daten kommen über den sogenannten Splicer in den Schleifenrumpf von ForEach und werden am Ende über den Accumulator ausgeleitet.

Den Mauscursor steuern Sie über das Leap-Motion-Gerät mit der Handfläche (= Ausgang „Palm Position“). Er liefert die Position als Vektor mit 3 Koordinaten (x, y, z). Außerhalb der ForEach-Schleife spaltet ein „`Vector (Split) [3D.Vector3]`“-Node diesen Vektor in seine einzelnen Komponenten auf. (Den „*Decons*“-Node brauchen Sie, um einen Spread in einen Vector umzuwandeln; hat der „*Decons*“-Node keinen Ausgang, markieren Sie den Node und fügen Sie mit STRG + „+“ einen hinzu.)




– **Kinect:** Damit vvvv die Kinect erkennt, muss zusätzlich zur Library das „Kinect for Windows Runtime v1.8 SDK“ installiert sein (auf den Laborrechnern bereits erledigt). Die vvvv-Library „Kinect“ installieren Sie über die Konsole („Quad menu“ → Manage Nugets → Commandline) mit `nuget install VL.Devices.Kinect`. Danach binden Sie sie über den NodeBrowser in den Patch ein.

Der „*Skeletons*“-Node der Kinect-Bibliothek liefert einen Spread. Die Palm-Position extrahieren Sie über ein „`Split.Devices.Kinect.Skeleton`“ deshalb in einem ForEach (rechte Maustaste / Surround). Die Daten kommen über den Splicer in das ForEach und werden über den Accumulator ausgeleitet. Außerhalb des ForEach wird der Spread in seine Outputs (Vector3) dekonstruiert und über einen „*Vector (Split)*“ in seine Komponenten aufgespalten.




– Implementieren Sie mit diesen Informationen die ursprüngliche Funktion des Patches (linearer Positionsgeber mit dem Mausrad) mit einem **Joystick**, einer **Leap Motion** bzw. einer **Kinect**. Sie werden sehen: Weil die Wertebereiche von Ein- und Ausgabe schon getrennt sind, lassen sich diese Sensoren ohne weitere Programmänderungen einbinden. Den linken bzw. rechten Rand stellen Sie genauso wie bei der Maus mit der Pfeiltaste links bzw. rechts ein.

**Frage 12:** Berichten Sie von Ihren Erfahrungen, Schwierigkeiten und „A-ha"-Effekten bei dieser Aufgabe.
*Dokumentieren Sie Ihre Patch-Änderungen textuell sowie z. B. mit Screenshots im Online-Protokoll (Moodle).*

## 3) Eine „sinnvolle“ Anwendung – Interaktion mit „Points of Interest“ in einem Foto/Hintergrundbild

Hier setzen Sie den gerade erstellten Patch „`Mausrad4.vl`“ in einem größeren Kontext ein: Sie steuern damit die Position in einem Foto, fahren „Points of Interest“ (POIs) an und lassen sich dazu Detailinformationen anzeigen. Natürlich können Sie auch die abgewandelten Versionen von „`Mausrad4.vl`“ verwenden und über Joystick, Leap Motion, Kinect usw. steuern. Im Folgenden geht es nur um die Basisvariante mit der Maussteuerung.

### 3.a) Vorbereitung: Kennenlernen des Projektes „PointsOfInterest“

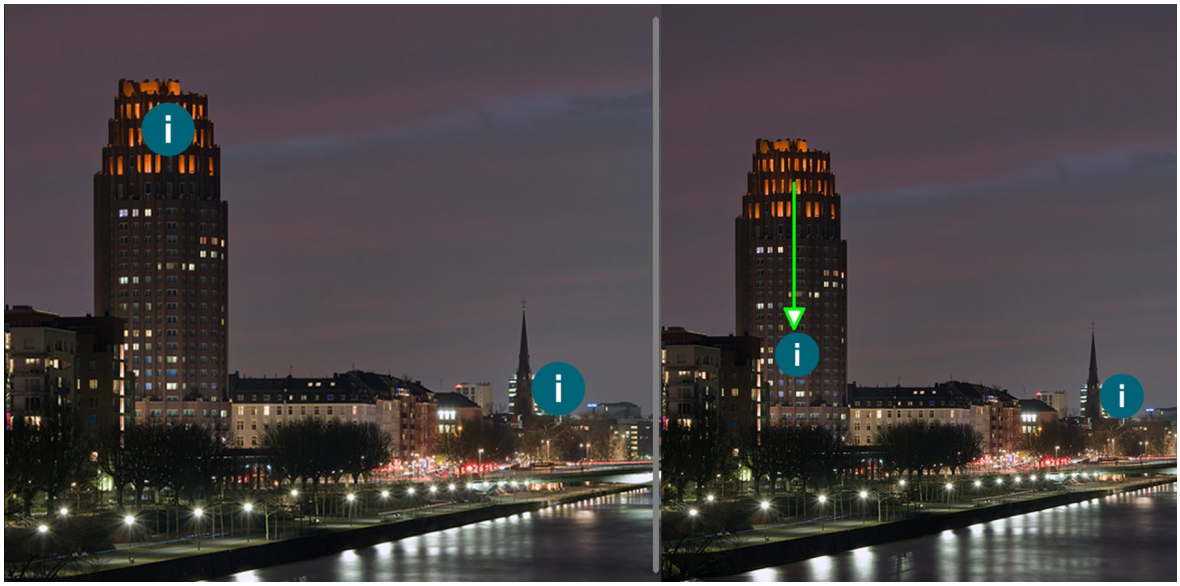
Laden Sie den Ordner „PointsOfInterest“ vom Laufwerk „P:“ (Datentausch) oder aus Moodle und öffnen Sie den Patch mit dem Dateinamen „`root.vl`“. Dieser Patch ist sozusagen das „Hauptprogramm“. Er greift auf andere Patches („Hilfsklassen“) zu, unter anderem auf den gerade erstellten Patch „`Mausrad4.vl`“ (hier: „Mausr“) und auf „GUI“.

Machen Sie sich mit diesem „*vvvv*“-Programm und seinen Subpatches vertraut und beantworten Sie folgende Fragen:

**Frage 13:** Wofür steht der Parameter „Epsilon“ im Patch „`root.vl`“? Was ändert sich, wenn Sie ihn auf 0.0 bzw. auf 0.15 ändern?

*Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).*

– Verschieben Sie das „i im Kreis“ (es zeigt vorhandene Zusatzinformation zu einem „Point of Interest“) für den „Main Plaza“ (=linker beleuchteter Turm im Bild) von der Spitze des Turms irgendwo in den Bereich des 8. Stockwerks, wie unten dargestellt:



**Frage 14:** Was mussten Sie am Patch ändern, um das „i“ zu verschieben?

*Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).*

### 3.b) Anpassen des Patches „PointsOfInterest“ für eine eigene Applikation

Als Nächstes nutzen Sie dieses „*vvvv*“-Programm als Basis für eine ähnliche Applikation und passen es an. Im Datentausch-Laufwerk „P:“ finden Sie einen Dateiordner „assetsTHI“. Kopieren Sie ihn in das Projektverzeichnis von „PointsOfInterest“. Sehen Sie sich die Dateien dieses Ordners und des ursprünglichen Ordners „assets“ an und vergleichen Sie die Inhalte. „assetsTHI“ enthält u. a. ein Foto des Hochschulcampus. Betten Sie es statt der Skyline von Frankfurt als Basis ein. Im Ordner liegen außerdem schon Grafiken für die „Hot Spots“ bzw. „Points of Interest“; platzieren und verlinken Sie auch diese (korrekt). Verwenden Sie folgende POIs: 1) Kavalier Dallwigk, 2) Audi Akademie, 3) Gießereihalle, 4) Hochschule, D, 5) Hochschule, G-Gebäude, 6) CARISSMA, 7) THI-Erweiterung.

Am Ende sollte das Ergebnis etwa so aussehen:




**Frage 15:** Was mussten Sie alles ändern, um zu dieser Lösung zu kommen? Welche Schwierigkeiten hatten Sie auf dem Lösungsweg? *Dokumentieren Sie Ihre Lösung bzw. den Lösungsweg als Text, machen Sie Screenshots und betten Sie sie in Ihr Protokoll ein.*

**BITTE BEACHTEN: Am Ende der Praktikumsstunde…**

- …verpacken Sie sämtliche Geräte (Joysticks, Kinects, Leap Motions) wieder so, wie Sie sie zu Beginn vorgefunden haben.
- …verlassen Sie die Arbeitsplätze sauber und gereinigt.